13.2. Graphs Traversal

Previous Section:

13.1. Representation of Graphs

Contents:

1. Traversal using Depth First Search (DFS)

2. Traversal using Breadth First Search (BFS)

3. Graph Traversal with Python

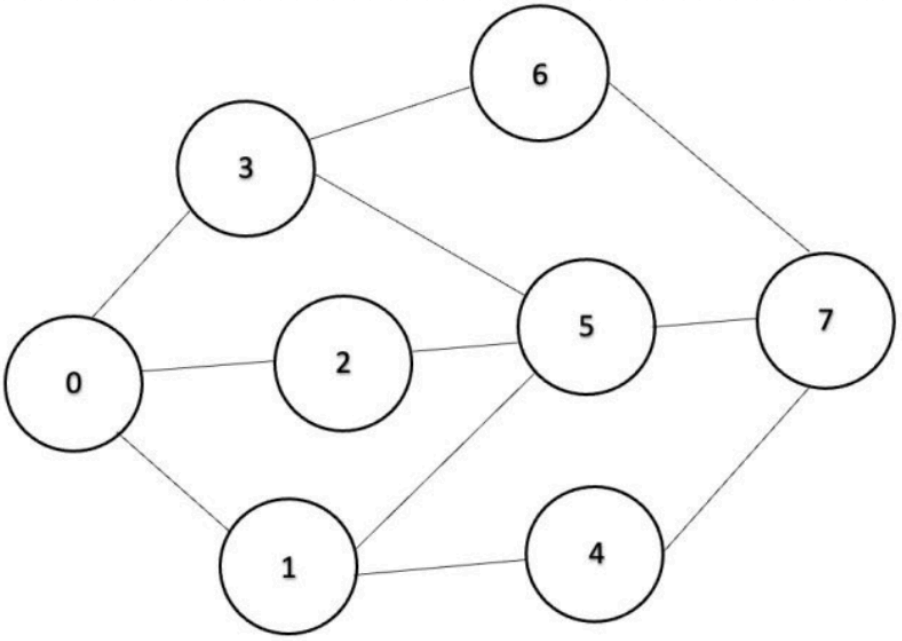
Summary

References

After learning how graphs are represented, the next step is to learn how we can **visit every vertex** in a graph. This process is known as **graph traversal**.

Graph traversal is one of the most fundamental operations in graph theory. Many advanced graph algorithms, such as shortest path algorithms, minimum spanning trees, and cycle detection, begin by traversing the graph.

Throughout this section, we will use the following graph as our example.



1. Traversal using Depth First Search (DFS)

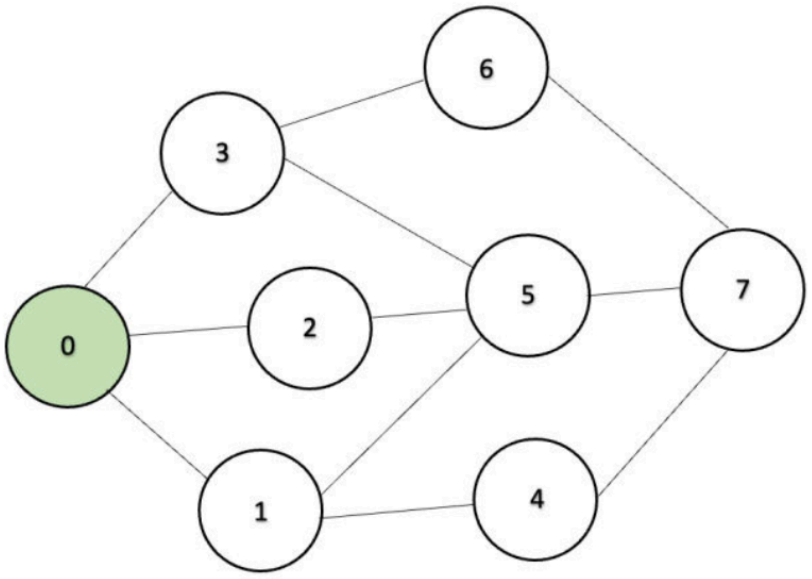
Depth First Search (DFS) explores a graph by visiting one branch as deeply as possible before returning to explore another branch.

Instead of visiting every neighboring vertex immediately, we continue moving deeper until the current vertex has no unvisited neighbors. At that point, we **backtrack** to the previous vertex and continue exploring the remaining branches.

DFS uses a **Stack**, so the **last vertex added is always visited first (LIFO)**.

Suppose we start from vertex **0**.

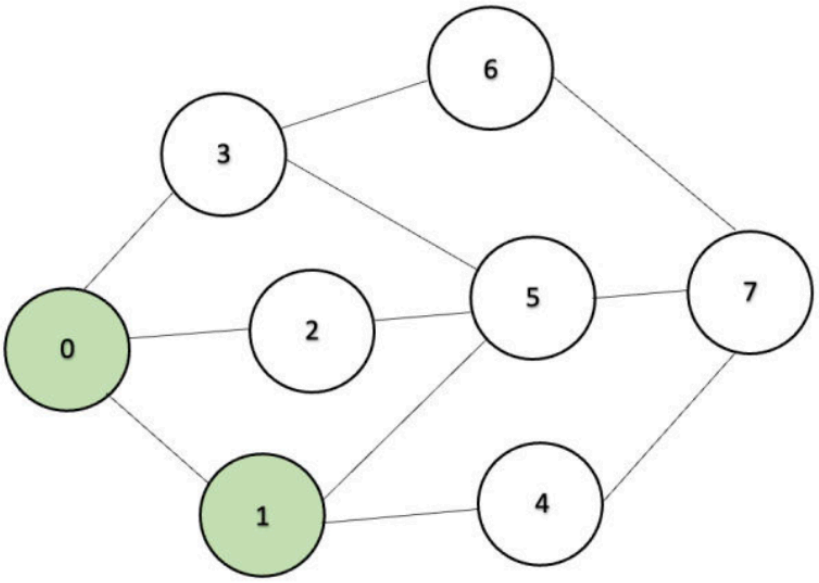
- Step 1 – Visit the starting verte




```
Current : 0

Stack (Top)
      ↓
    +---+
    | 1 |
    +---+
    | 2 |
    +---+
    | 3 |
    +---+

Visited : [0]
```

We begin by adding 0 to the visited list, and pushing the neighbors onto the stack.

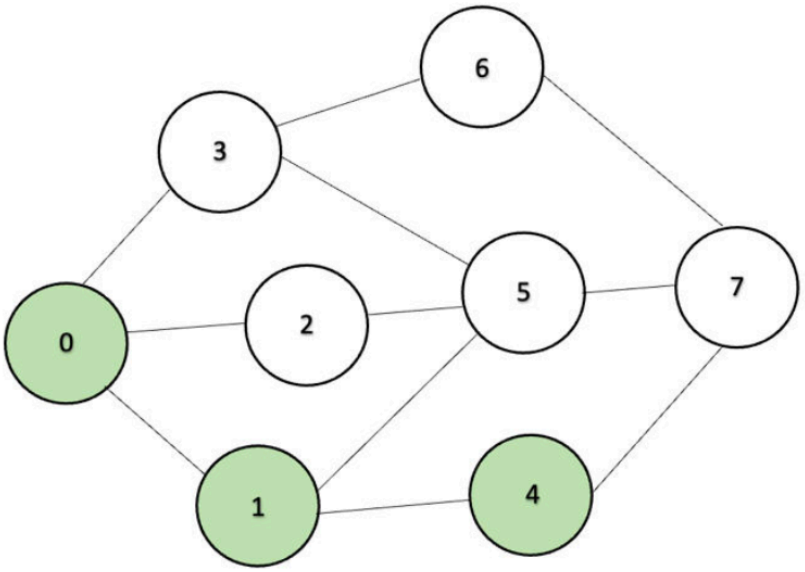
- Step 2 – Move to vertex




```
Current : 1

Stack (Top)
      ↓
    +---+
    | 4 |
    +---+
    | 5 |
    +---+
    | 2 |
    +---+
    | 3 |
    +---+

Visited : [0, 1]
```

As shown, vertex **0** has several neighbors. We choose vertex **1 by popping** it out of the stack, then pushing the neighbors of 1 onto the stack, and continue exploring deeper.

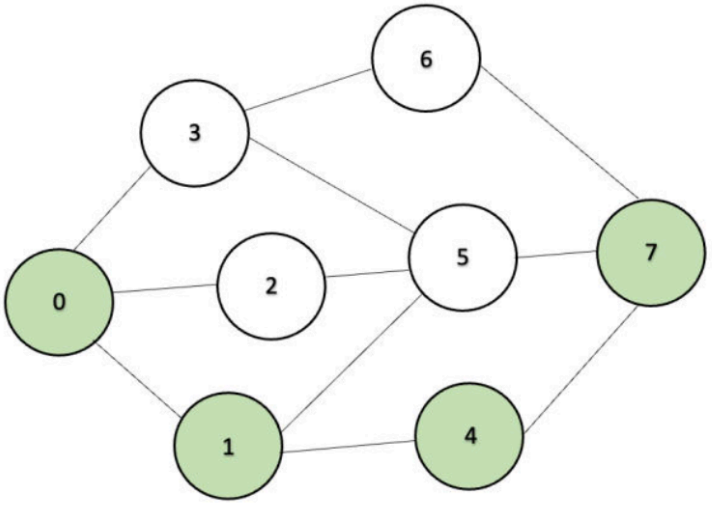
- Step 3 – Move to vertex 4




```
Current : 4

Stack (Top)
      ↓
    +---+
    | 7 |
    +---+
    | 5 |
    +---+
    | 2 |
    +---+
    | 3 |
    +---+

Visited : [0, 1, 4]
```

Here we continue following the current path instead of exploring other branches.

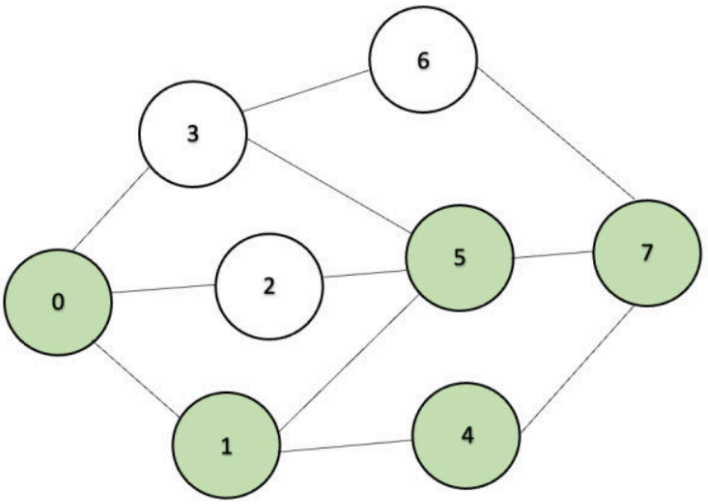
- Step 4 – Move to vertex 7




```
Current : 7

Stack (Top)
      ↓
    +---+
    | 5 |
    +---+
    | 2 |
    +---+
    | 3 |
    +---+

Visited : [0, 1, 4, 7]
```

DFS continues extending the current branch.

- Step 5 – Move to vertex



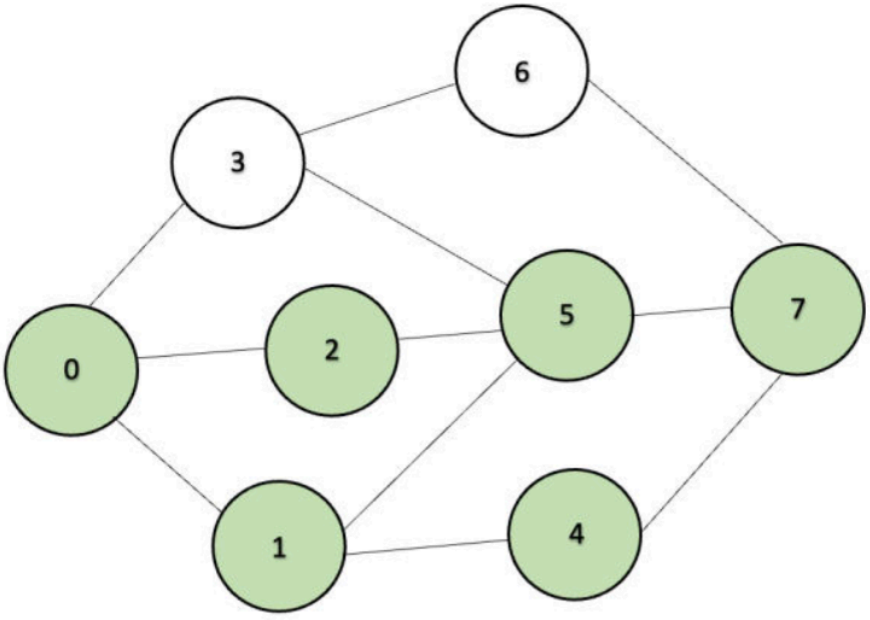

```
Current : 5

Stack (Top)
      ↓
    +---+
    | 2 |
    +---+
    | 3 |
    +---+

Visited : [0, 1, 4, 7, 5]
```

Vertex **5** still has unvisited neighbors, so DFS continues deeper.

Note: Although vertex **7** is adjacent to both **5** and **6**, DFS does not visit both neighbors immediately.
Instead, it continues exploring **one unvisited neighbor at a time**.

- In this example, vertex **5** is chosen first, and DFS explores the entire branch starting from **5** before returning to vertex **7**.
- Only after all vertices reachable through **5** have been visited does DFS return to **7** and continue with its remaining unvisited neighbor, **6**.
- This "go as deep as possible before backtracking" behavior is the defining characteristic of Depth First Search.

- Step 6 – Reach vertex 2

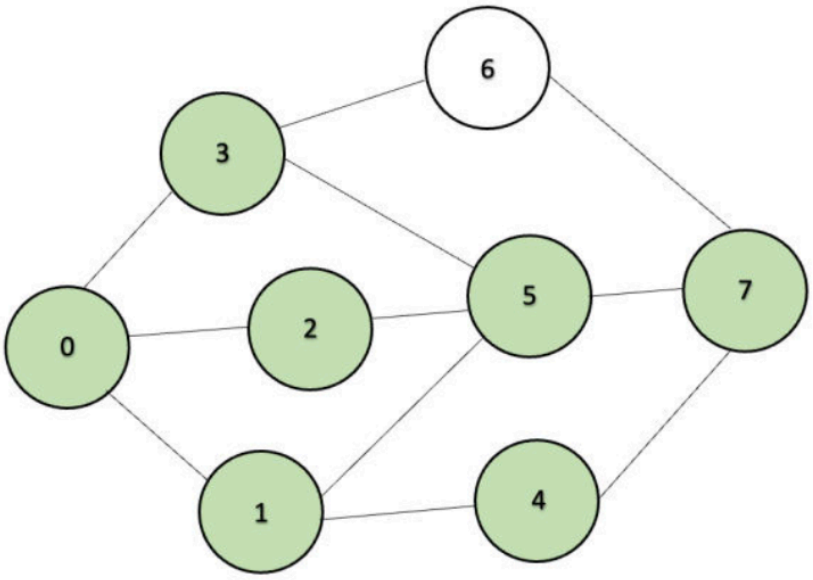



```
Current : 2

Stack (Top)
      ↓
    +---+
    | 3 |
    +---+

Visited : [0, 1, 4, 7, 5, 2]
```

Vertex **2** has no unvisited neighbors. Therefore, DFS has **backtracked** to vertex **5**.

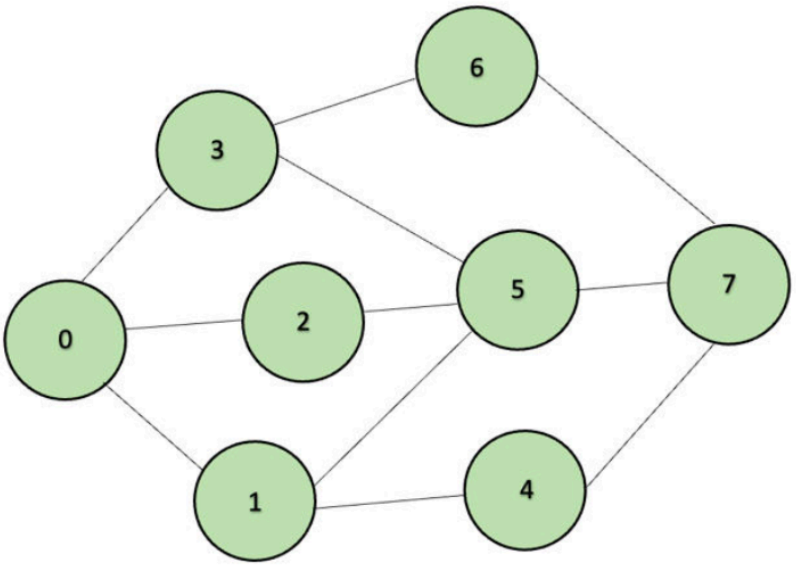
- Step 7 – Continue exploring

Since vertex **5** still has another unvisited neighbor (**3**), we continue exploring.




```
Current : 3

Stack (Top)
      ↓
    +---+
    | 6 |
    +---+

Visited : [0,1,4,7,5,2,3]
```

- Step 8 – Visit the final vertex




```
Current : 6

Stack becomes empty

Visited : [0,1,4,7,5,2,3,6]
```

All vertices have now been visited.

One possible DFS traversal is `[0, 1, 4, 7, 5, 2, 3, 6]`

The time complexity of DFS is $O(V + E)$ because every vertex and every edge is explored at most once.

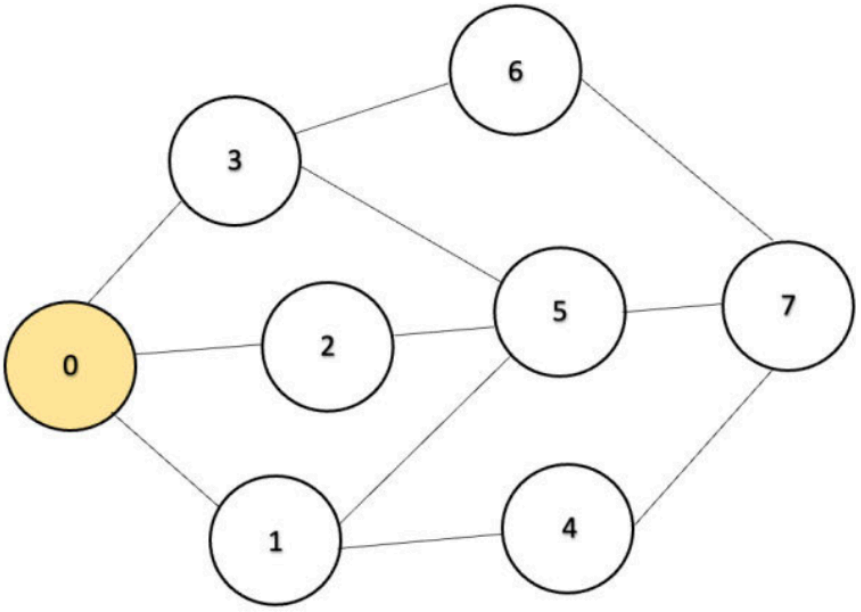
2. Traversal using Breadth First Search (BFS)

Breadth First Search (BFS) explores a graph one **level at a time**.

Instead of following one branch deeply, BFS first visits every neighboring vertex before moving farther away from the starting vertex.

BFS uses a **Queue**, so the **first vertex added is always visited first (FIFO)**.

- Step 1 – Start from vertex 0

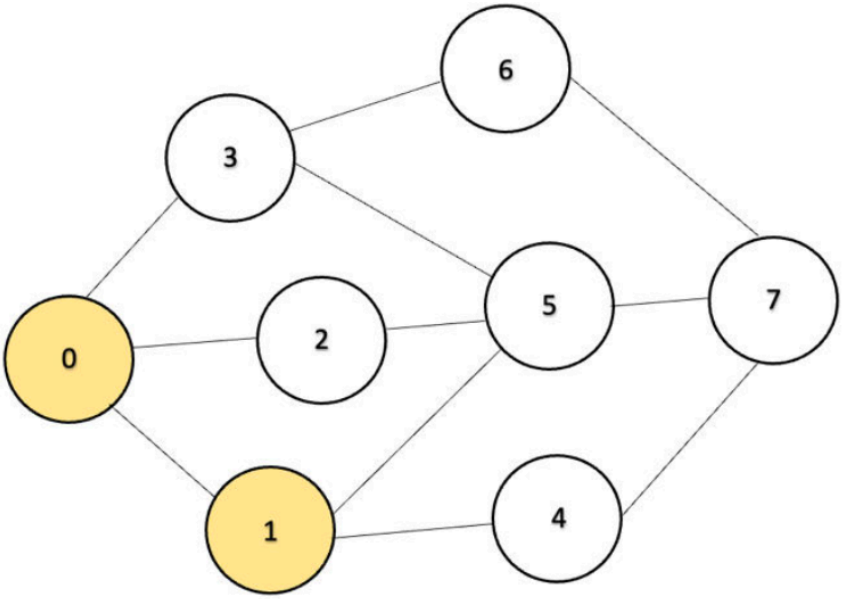



```
Current : 0

Queue (Front)
      ↓
+---+
| 0 |
+---+

Visited : []
```

- Step 2 – Enqueue all neighbors of 0

```
Queue (Front)
      ↓
+---+---+---+
| 1 | 2 | 3 |
+---+---+---+

Visited : [0]
```

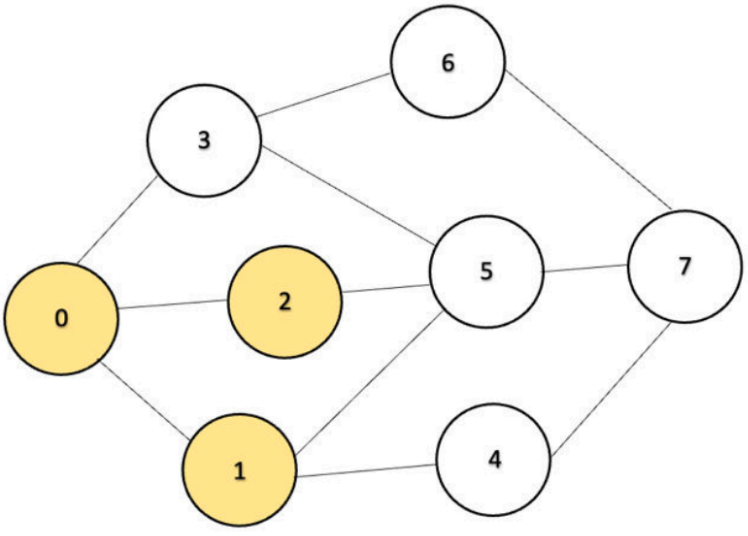
- Step 3 – Process vertex 1



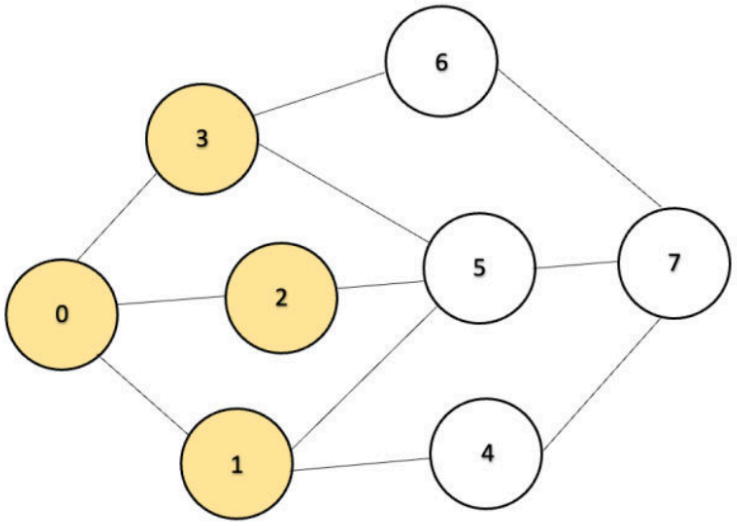

```
Current : 1

Queue (Front)
      ↓
+---+---+---+---+
| 2 | 3 | 4 | 5 |
+---+---+---+---+

Visited : [0,1]
```

- Step 4 – Continue processing

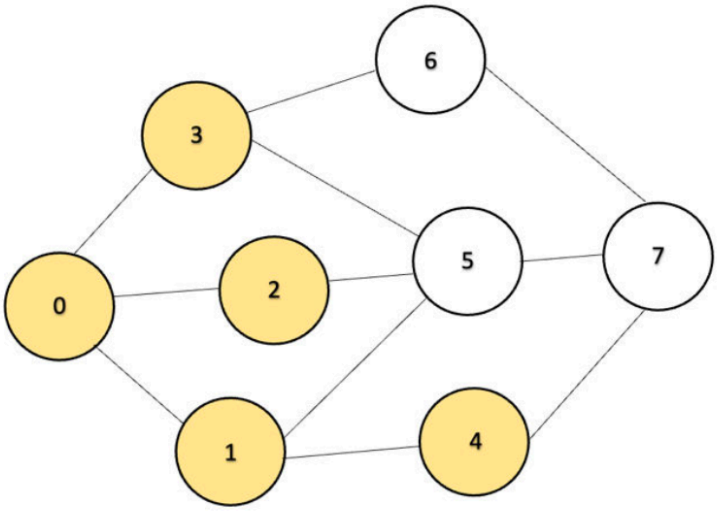
After processing vertices **2**, **3**, and **4**, vertex **6** and then **7** are discovered.



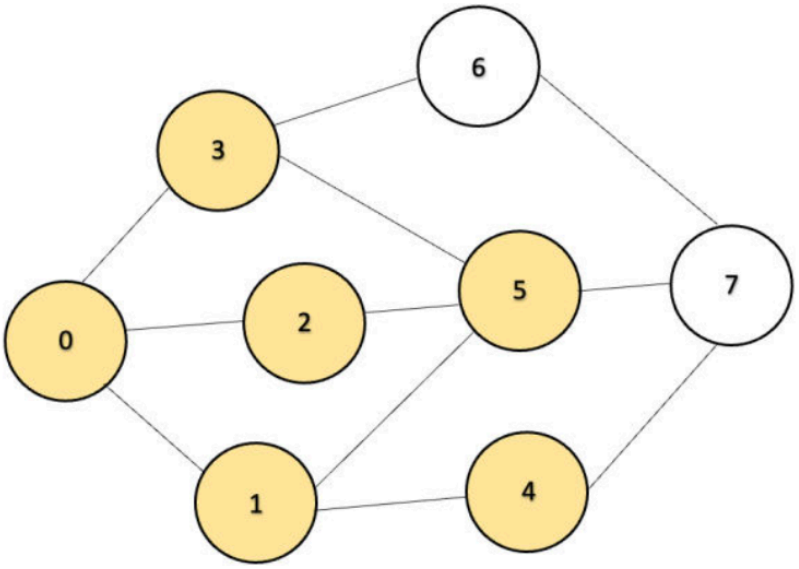
```
Current : 2

Queue (Front)
      ↓
+---+---+---+---+
| 3 | 4 | 5 |   |
+---+---+---+---+

Visited : [0,1,2]
```




```
Current : 3

Queue (Front)
      ↓
+---+---+---+---+
| 4 | 5 | 6 |   |
+---+---+---+---+

Visited : [0,1,2,3]
```

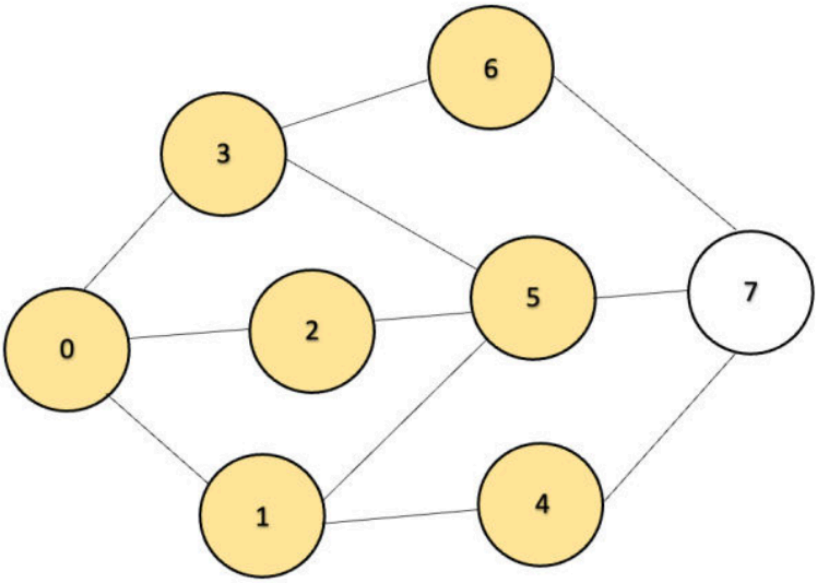



```
Current : 4

Queue (Front)
      ↓
+---+---+---+---+
| 5 | 6 | 7 |   |
+---+---+---+---+

Visited : [0,1,2,3,4]
```

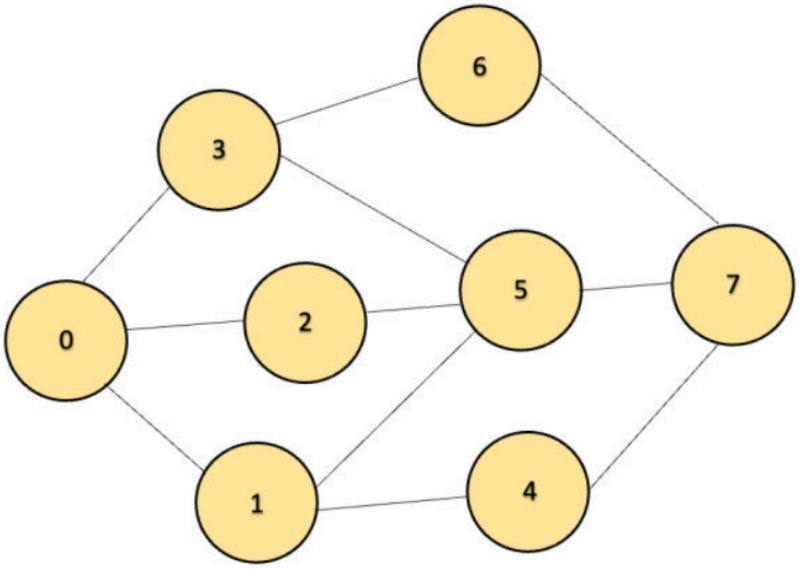



```
Current : 5

Queue (Front)
```

```
      ↓
+---+---+---+---+
| 6 | 7 |   |   |
+---+---+---+---+

Visited : [0,1,2,3,4,5]
```




```
Current : 6

Queue (Front)
      ↓
+---+---+---+---+
| 7 |   |   |   |
+---+---+---+---+

Visited : [0,1,2,3,4,5,6]
```




```
Current : 7

Queue (Front)
      ↓
+---+---+---+---+
|   |   |   |   |
+---+---+---+---+

Visited : [0,1,2,3,4,5,6,7]
```

One possible BFS traversal is `[0,1,2,3,4,5,6,7]`

Like DFS, BFS also has a time complexity of $O(V + E)$.

3. Graph Traversal with Python

Since we already introduced a `Graph` class in the previous section, it is much cleaner to make **DFS** and **BFS** become **methods of the Graph class** rather than standalone functions.

This also allows both algorithms to directly use the adjacency matrix stored in the object.

```
class Graph:

    ...

    # Depth First Search (Recursive)
    def depth_first_search(self, node, visited=None):
        if visited is None:
            visited = set()

        print(node, end=" ")
        visited.add(node)

        for neighbor in range(self.vertices):
            if self.matrix[node][neighbor] == 1 and neighbo
r not in visited:
                self.depth_first_search(neighbor, visited)

    # Breadth First Search
    def breadth_first_search(self, start):
        visited = set()
        queue = []

        visited.add(start)
        queue.append(start)

        while queue:
            node = queue.pop(0)
            print(node, end=" ")

            for neighbor in range(self.vertices):
                if self.matrix[node][neighbor] == 1 and nei
ghbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
```

```
graph = Graph(8)

graph.add_edge(0, 1)
graph.add_edge(0, 2)
graph.add_edge(0, 3)
graph.add_edge(1, 4)
graph.add_edge(1, 5)
graph.add_edge(2, 5)
graph.add_edge(3, 5)
graph.add_edge(3, 6)
graph.add_edge(4, 7)
graph.add_edge(5, 7)
graph.add_edge(6, 7)

print("Adjacency Matrix:")
graph.display()

print("\nDepth First Search:")
graph.depth_first_search(0)

print("\nBreadth First Search:")
graph.breadth_first_search(0)
```

```
Adjacency Matrix:
[[0 1 1 1 0 0 0 0]
[1 0 0 0 1 1 0 0]
[1 0 0 0 0 1 0 0]
[1 0 0 0 0 1 1 0]
[0 1 0 0 0 0 0 1]
[0 1 1 1 0 0 0 1]
[0 0 0 1 0 0 0 1]
[0 0 0 0 1 1 1 0]]

Depth First Search:
0 1 4 7 5 2 3 6
Breadth First Search:
0 1 2 3 4 5 6 7
```

Summary

Graph traversal provides a systematic way to visit every vertex in a graph.

In this section, we explored two traversal methods:

- **Depth First Search (DFS)** uses a **Stack** and explores one path as deeply as possible before backtracking.
- **Breadth First Search (BFS)** uses a **Queue** and explores the graph level by level.

Although the traversal orders are different, both algorithms visit every reachable vertex efficiently with a time complexity of **O(V + E)**. These traversal techniques serve as the foundation for many advanced graph algorithms studied later.

References

https://drive.google.com/file/d/1R4R6eGz_XZEva12XTJEv3olhZt1aet4x/view?usp=drive_link

https://www.youtube.com/watch?v=pcKY4hjDrxk

Next Section:

13.3. Insertion and Deletion in Graphs